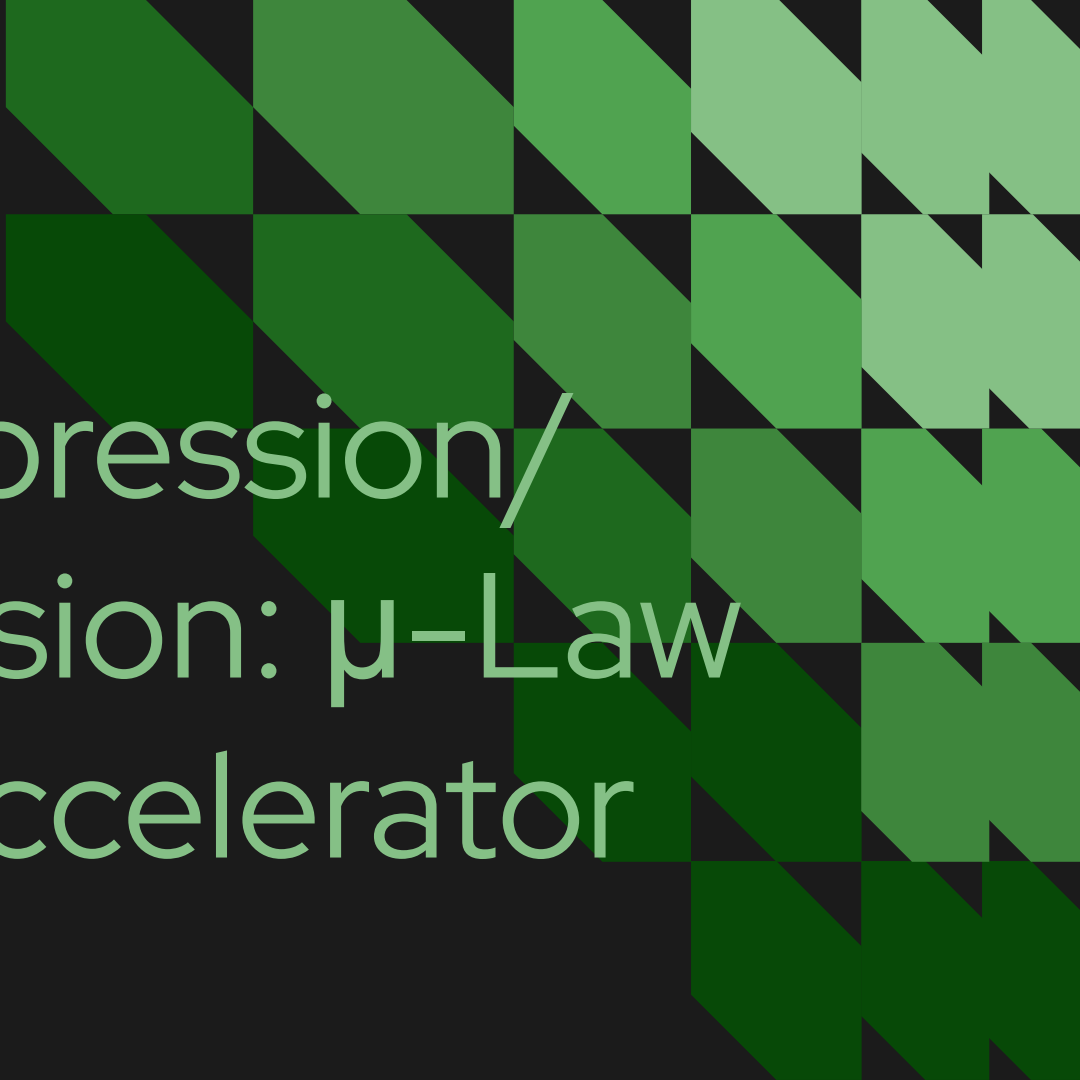


An abstract pattern of overlapping green squares and rectangles of various shades, creating a sense of depth and movement, located on the right side of the slide.

Audio Compression/ Decompression: μ -Law Hardware Accelerator

Presenters: Manraj Minhas & Julian Lo

Introduction & Motivation

The Objective

Reduce data bandwidth for digital telephony while preserving speech intelligibility.

The Standard

Implement the North American μ -law companding algorithm.

- Compresses 14-bit linear audio samples into 8-bit codewords.

The Bottleneck

Software-based logarithmic approximations introduce unacceptable latencies in real-time embedded systems.

Proposed Solution

A hardware/software co-design targeting a 32-bit ARM processor to accelerate the μ -law computation.

Acoustic Theory & PCM

Characteristics of Speech

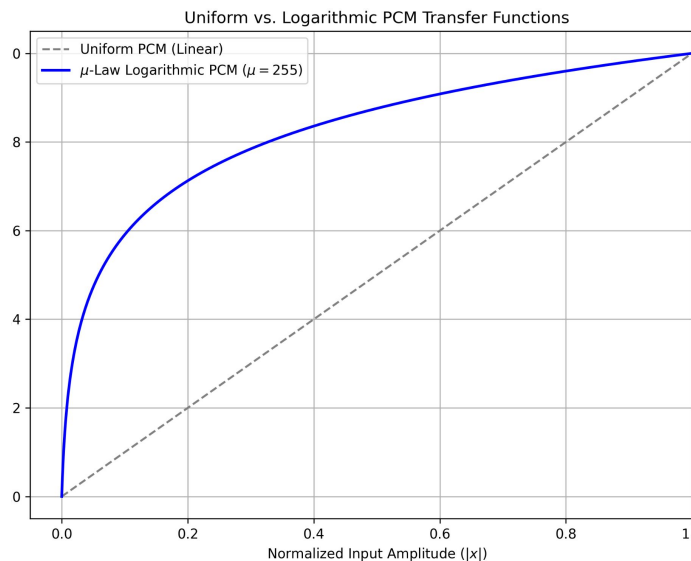
- **Voiced Phonemes (Vowels):** High amplitude, occur frequently, but carry less information.
- **Unvoiced Phonemes (Consonants):** Low amplitude, but contain critical acoustic information.

Logarithmic PCM (μ -Law)

- Matches the human auditory system logarithmic transfer function.
- Maintains a constant relative quantization error across the dynamic range.

Uniform PCM Inefficiency

- Applies equal quantization intervals, wasting resolution on loud signals.



Baseline Software Bottlenecks

Fixed-Point Math & Biasing

- A constant bias of 33 is added to the signal magnitude.
- Mathematically forces chord boundaries to align perfectly with powers of 2.

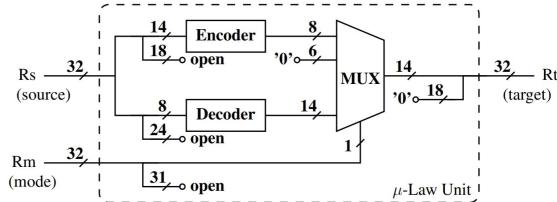
The Software Implementation

- Finding the chord reduces to locating the most significant '1' bit.
- Implemented in C using a massive cascade of sequential if-then-else statements.

The Processor Bottleneck

- **Branching Overhead:** Unpredictable if-statement branching causes severe ARM pipeline flushes.
- **Inefficiency:** Checks largest values first, artificially inflating instruction count for low-amplitude speech.
- **Latency:** Demands worst-case execution time of ~25 clock cycles per sample.

Custom Hardware Datapath



Architectural Extension

Offloaded computation to a dedicated hardware functional unit via a custom μ Law instruction.

Logic Replacement

Replaced sequential software branching with pure combinational hardware.

Hardware Latency

Operation completes in just 3 clock cycles.

Key Datapath Components

- **Priority Encoder:** Instantly determines the position of the most significant '1' (Chord calculation).
- **Barrel Shifter:** Extracts the variable step bits in a single action.
- **Multiplexer (MUX):** Toggles between hardware Encoder and Decoder blocks using the 1-bit mode register.

Software- Hardware Integration

Custom ARM Instruction

Extended the Instruction Set Architecture (ISA) with a custom muLaw instruction to call the hardware block.

The Initial Flaw

- Standard inline assembly forced compiler to push variables to stack memory.
- Generated massive Load/Store (LDR/STR) bottleneck, negating speedup.

The Compiler Solution

- Utilized the C register specifier (`register int sample`).
- Forced compiler to keep variables entirely within CPU internal registers.

Result

Completely bypassed stack memory, establishing a highly optimized, lean, inline execution sequence.

Verification Methodology

Virtualized Testing Environment

- Utilized a QEMU-based Linux virtual machine configured for the ARM Hard Float 32-bit (ARMHF32) architecture.
- Ensured highly accurate simulation of instruction sets, memory constraints, and pipeline behavior.

Data Preparation

- Recorded a 10-second uncompressed speech baseline as a 16-bit PCM .wav file (8000 Hz, Mono).

Header Repair Strategy

- Standardized WAV headers are exactly 44 bytes, but OS metadata injection corrupted output files.
- Developed a Python extraction script (fix_audio.py) to bypass variable-length headers and perfectly reconstruct the audio structure.



Audio Fidelity & Quantization Error

Objective

Verify that the 1.75:1 compression ratio successfully preserved speech intelligibility.

Acoustic Profile

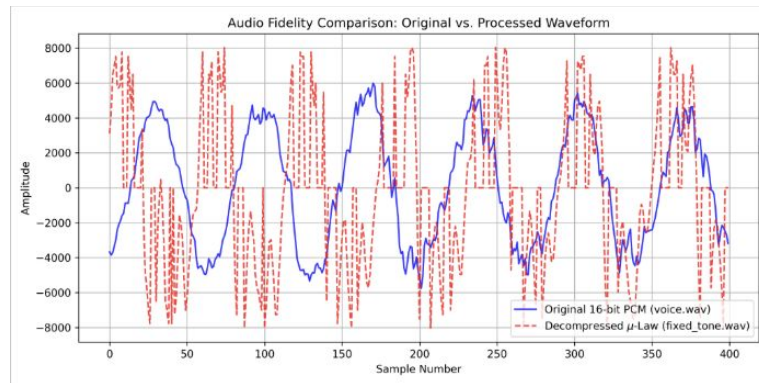
- Successfully reconstructed the original 10-second voice sample.
- Introduced the expected telephony "buzz" characteristic of non-uniform step sizes.

Quantization Error Analysis

- Small signals (consonants) lose zero or few bits, preserving critical detail.
- Large signals (vowels) lose up to 6 bits of precision, creating absolute error biologically masked by human hearing.

Audio Fidelity Comparison

Original 16-bit PCM vs. Decompressed μ -Law Waveform



Performance Results & Cycle Count

Baseline Software Profile

25 Cycles

Worst-case execution time due to sequential branching.

Hardware Latency

3 Cycles

Pure combinational logic of the custom datapath.

Optimized Integration

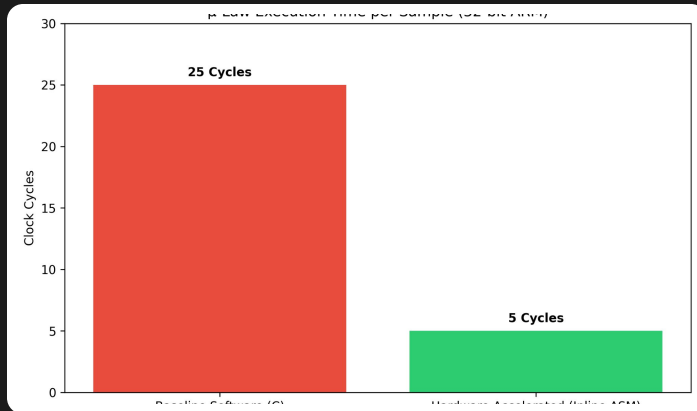
5 Cycles

Total inlined software-hardware execution time.

Final Speedup

5X

Massive architectural speedup over the C baseline.



System Impact

Eliminated branch mispredictions and pipeline stalls, yielding a highly deterministic and power-efficient solution.

Conclusion & Future Scalability

Project Success

Successfully implemented a real-time μ -law codec on a 32-bit ARM processor, reducing data bandwidth while preserving intelligibility.

Future: Compatibility

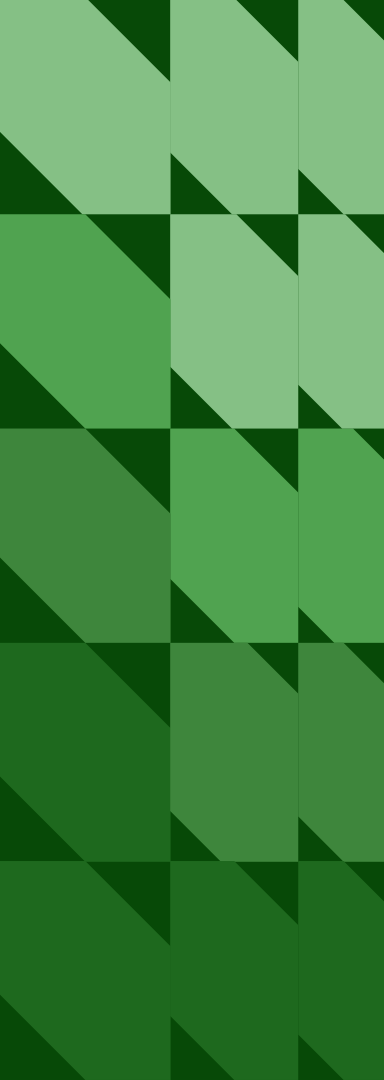
Add a control bit to toggle between μ -law and European A-law companding blocks.

Future: Stereo SIMD

Pack Left and Right 16-bit audio channels into a single 32-bit register to double data throughput using simultaneous processing.

Future: Prototyping

Transition from virtualized QEMU simulation to physical FPGA deployment for precise electrical power profiling.



Thank you

Any Questions?